Università degli Studi di Napoli
Federico II
Dipartimento di Ingegneria Elettrica e delle Telecomunicazioni

Statistical Learning and Data Analysis – Module A

INTRODUCTION TO PROGRAMMING

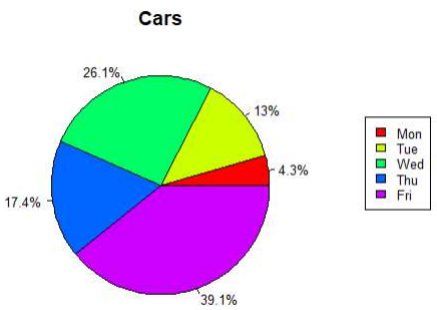
Prof. Emiliano del Gobbo

emiliano.delgobbo@unina.it

1

What we can do with R

```{r}
# Definisce un vettore di auto
cars <- c(1, 3, 6, 4, 9, 2)

# Definisce i colori
colors <- rainbow(length(cars))
#colors <- c("white","grey70","grey90","grey50","black")

# Calcola le percentuali arrotondate al primo decimale
car_labels <- round(cars/sum(cars) * 100, 1)

# Concatena il simbolo '%' dopo ogni valore
car_labels <- paste(car_labels, "%", sep="")

# Crea il grafico a torta
pie(cars, main="Vendita automobili della settimana", col=colors, labels=car_labels,
    cex=0.8)

# Mostra la leggenda sulla destra
legend(1.5, 0.5, c("Mon","Tue","Wed","Thu","Fri"), cex=0.8,
   fill=colors)
```

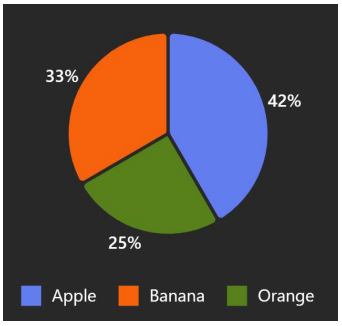


© 2025 – Prof. Emiliano del Gobbo

2

What we can do with Python

```python
import pandas as pd
import plotly.express as px

# Example data: survey responses by favorite fruit
survey_data = {
    "Fruit": ["Apple", "Banana", "Orange", "Apple", "Banana", "Apple",
              "Orange", "Banana", "Apple", "Banana", "Orange", "Apple"]
}
df = pd.DataFrame(survey_data)

# Calculate frequency of each category
freq = df["Fruit"].value_counts().reset_index()
freq.columns = ["Fruit", "Count"]

# Plot a pie chart
fig = px.pie(freq, names="Fruit", values="Count", title="Favorite Fruit Survey Results")
fig.show()
```



© 2025 – Prof. Emiliano del Gobbo

Why R?

- R is a free and open-source software environment, in contrast to proprietary platforms such as Matlab, Stata, or SPSS.
- It is cross-platform and fully compatible with Windows, Linux, and MacOS.
- As a scripting language rather than a simple graphical interface-based program, R provides a high degree of flexibility in data analysis and programming.
- It benefits from a broad and active user community, which facilitates the availability of extensive documentation and numerous examples of code.
- Moreover, R offers a comprehensive ecosystem of libraries supporting a wide range of statistical, analytical, and computational tasks.
- Its relatively low entry barrier makes it accessible even to individuals without prior programming experience.
- Finally, R includes advanced graphical libraries that enable the production of high-quality visualizations in an efficient manner.

© 2025 – Prof. Emiliano del Gobbo

Why Python?

- **Free and Open Source:** Python is freely available and supported by a vast global community.
- **Cross-Platform Compatibility:** Runs seamlessly on Windows, Linux, and MacOS.
- **General-Purpose & Versatile:** Suitable for data analysis, machine learning, web development, automation, and more.
- **Readable Syntax:** Intuitive and easy-to-learn, making it accessible for beginners and efficient for experts.
- **Rich Ecosystem of Libraries:** Extensive libraries for data science (NumPy, pandas), machine learning (scikit-learn, TensorFlow), and visualization (matplotlib, seaborn, Plotly).
- **Active Community:** Abundant tutorials, forums, and documentation available for all skill levels.
- **Integration Capabilities:** Easily integrates with other languages and tools (e.g., R, SQL, C/C++, Java).
- **High-Quality Visualizations:** Advanced plotting libraries enable the creation of publication-ready graphics.

© 2025 – Prof. Emiliano del Gobbo

R vs Python

Feature	R	Python
Origin & Focus	Designed for statistics and data analysis	General-purpose language, strong in data science
Ease of Use	Intuitive for statisticians, especially with formulas	Readable syntax, easy for beginners and programmers
Statistical Packages	Extensive built-in and third-party statistical packages	Rich ecosystem (e.g., pandas, statsmodels, scikit-learn)
Visualization	Advanced graphics (ggplot2, lattice, base plotting)	Powerful libraries (matplotlib, seaborn, plotly)
Community	Large, active, academic/statistical focus	Larger, diverse, cross-disciplinary community
Integration	Integrates well with statistical tools and databases	Integrates with web, databases, machine learning, etc.
Learning Curve	Steeper for programming, easier for statistical tasks	Gentle learning curve, especially for programming
Use Cases	Academic research, statistical modeling, bioinformatics	Data science, machine learning, automation, web apps

© 2025 – Prof. Emiliano del Gobbo

What does programming mean in computer science?

- Programming is the process undertaken by the programmer to design and implement a program that performs a specific task.
- The fundamental purpose of a program is that it *does* something—regardless of whether it is useful or not.
- In general terms, a program can be understood as receiving an input and producing an output. While this is not always the case, for the purposes of this course we will focus exclusively on programs of this type.

© 2025 – Prof. Emiliano del Gobbo

How to program?

Programming involves a set of specific actions:

- Analyzing the problem
- Designing an algorithm capable of solving it
- Encoding the algorithm into executable instructions

© 2025 – Prof. Emiliano del Gobbo

What is a problem

A problem is essentially a task that requires careful attention to be solved.

It can take many forms, for example:

- Performing a simple addition, such as 4 + 6.
- Solving an equation.
- Searching for a word within a text.
- Determining the most frequent word in a text.
- Compressing an image.

© 2025 – Prof. Emiliano del Gobbo

How to solve a problem

To develop a program that solves a problem, it is necessary to:

- Identify the available **input data**.
- Determinate the desired **output**.***
- Identify an appropriate procedure, i.e., an **algorithm**.
- Break down the procedure into a sequence of **elementary and unambiguous actions**.

***The desired output may not always be achievable, due to limitations such as inadequate input data, constraints of current technology, or computational resource limitations.

© 2025 – Prof. Emiliano del Gobbo

What is an algorithm?

An **algorithm** is a finite sequence of instructions that specifies how certain elementary operations must follow one another over time in order to solve a given class of problems.

© 2025 – Prof. Emiliano del Gobbo

How is an algorithm structured?

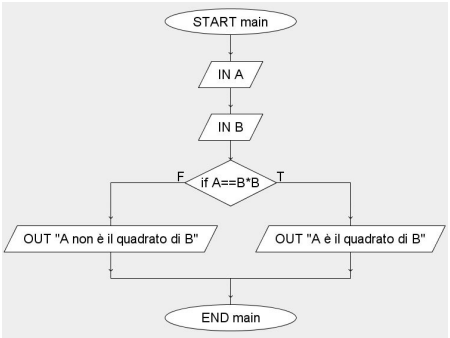
- **Instructions:** generally indicate requests for actions directed to the executor, which must be understood and carried out. Example: store a given value, jump to a specific instruction, display a result externally, etc.
- **Elementary operations:** refer to operations assumed to be known by the executor, and therefore executable by them. Example: addition and subtraction of integers, logical AND, etc.
- **Class of problems:** denotes a formulation of the problem that is independent of the specific data being operated on; for instance, not the sum of the numbers 34 and 57, but the sum of any two integers.

© 2025 – Prof. Emiliano del Gobbo

Algorithms as flowchart

The execution flow of an algorithm can be graphically represented using a flowchart:

- Eliminates ambiguities by employing a clear selection logic and unambiguous flow lines
- Relies on a small set of symbols
- Constitutes a universal language

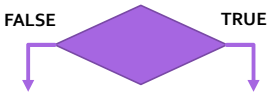


© 2025 – Prof. Emiliano del Gobbo

Main symbols of a flowchart

- **Ellipse:** represents the start and end of the flowchart.

- **Parallelogram:** represents the input or output data of the flowchart.



- **Diamond:** represents decision-making operations within the flowchart.

- **Rectangle:** represents computational or processing operations.

© 2025 – Prof. Emiliano del Gobbo

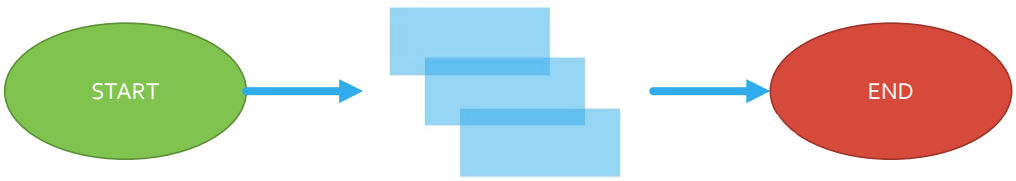
Flowchart: arrows

- **Arrows** are a fundamental element of the flowchart. They are used to connect various instructions and indicate the direction, that is, the program's flow.

© 2025 – Prof. Emiliano del Gobbo

Flowchart: Ellipse

Every program must always have a **start** and an **end**, and contain at least one instruction, which is therefore enclosed within **ellipses**.



© 2025 – Prof. Emiliano del Gobbo

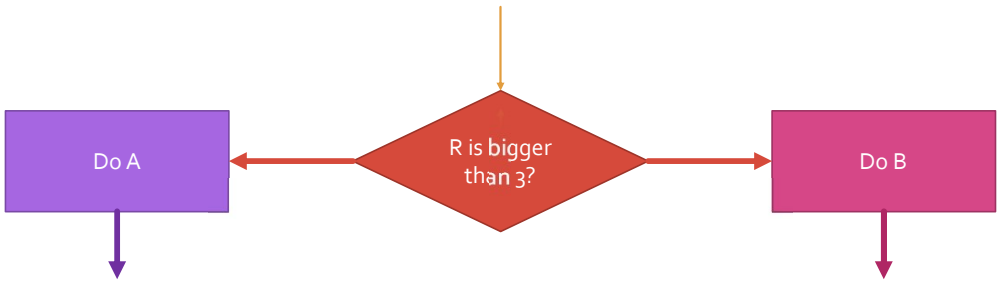
Flowchart: parallelogram

Parallelograms indicate **operations for reading variables, defining them, and finally returning the value of a variable**.

© 2025 – Prof. Emiliano del Gobbo

Flowchart: diamonds

- Diamonds define **conditional statements**. These are perhaps the most interesting, as they can alter the flow of the diagram. Indeed, depending on whether a given condition is satisfied, the diamond directs the flow in a different way.



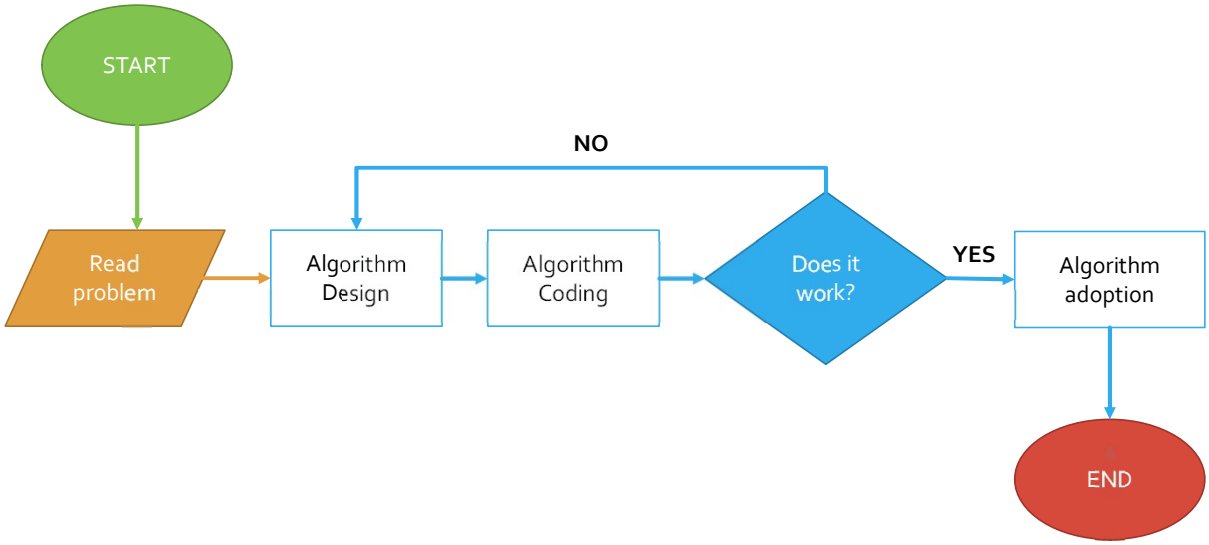
© 2025 – Prof. Emiliano del Gobbo

Flowchart: rectangles

Rectangles represent a **process** or an **action** that must be executed unconditionally once the flow reaches them.

© 2025 – Prof. Emiliano del Gobbo

The Programming Flowchart



© 2025 – Prof. Emiliano del Gobbo

Fundamental Components of a Computer

RAM Memory

Short-term memory is fast but is lost when power is removed. It stores the data currently in use by programs.

Processor

The brain of the computer. It executes the instructions provided, making use of memory and peripheral devices.

Storage Memory

Long-term memory is where we store our files and data. It is significantly slower than RAM.

© 2025 – Prof. Emiliano del Gobbo

Programming Languages

- **Languages? What language does the computer speak?**
- Naturally, as a machine, the computer does not speak human language at its most basic level. Instead, it uses **machine language**, encoded in binary—the well-known sequence: 010101010010…
- Therefore, to communicate with the computer, we would have to provide instructions in its own language, which is a highly complex task.
- This is where **programming languages** come to our aid.

© 2025 – Prof. Emiliano del Gobbo

Programming Languages 2

- To avoid the need to write directly in machine language, **interpreters** were developed. Their role is to translate programming languages that are closer to human language into machine language.
- C, C++, PHP, HTML, CSS, R, Python ecc…

© 2025 – Prof. Emiliano del Gobbo

Example

Pseudocode Example 7: Solve Quadratic Equation $ax^2 + bx + c = 0$

(Pseudocode If Else Example)

```

BEGIN
NUMBER a, b, c, d, x1, x2
INPUT a,b,c
d = b^2-4ac
IF (d >= 0) THEN
      x1 = (-b+√d)/2a yada x1 = (-b+d^(1/2)/2a
      x2 = (-b-√d)/2a yada x2 = (-b-d^(1/2)/2a
      OUTPUT "ROOT 1:"+x1
      OUTPUT "ROOT 2:"+x2
 ELSE IF (d == 0) THEN
      x1=x2= -b/2a
      OUTPUT "ROOT 1:"+x1
      OUTPUT "ROOT 2:"+x2
ELSE
    OUTPUT "There is no real root"
ENDIF
END

```

© 2025 – Prof. Emiliano del Gobbo

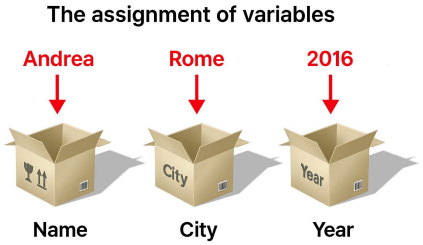
Variables

A **variable** is the fundamental element of programming. A variable represents a container that holds a value of any type.

Variables can be thought of as the "letters" in a literal mathematical expression.

They serve both as containers for stored values and as placeholders when referenced within an expression.

```
radius = 12
circumference = radius * 2 * 3.14
```



© 2025 – Prof. Emiliano del Gobbo

Type of variable

Variables can be of different types. The main ones are:

- **Numeric**
- **String**
- **List**
- **Object**

© 2025 – Prof. Emiliano del Gobbo

Numeric variable

A **numeric variable** is a variable that contains a number.
For example:

```
k = 5
z = 8,34
g = -1,56
```

© 2025 – Prof. Emiliano del Gobbo

String variable

A **string variable** is a variable that contains text. Typically, when defining a string variable, the text is enclosed in quotation marks.

work = "Insurer"

studenttype = "*commuter*"

Note!
For example:

```
a = 9.12
b = "9.12"
```

a and ***b*** are different, because one is a number and the other is a string. However, in some programming languages, the interpreter may automatically convert the type.

© 2025 – Prof. Emiliano del Gobbo

List Variable

The **List** variable is a type of variable that contains a collection of elements, which may include strings, numbers, other lists, or objects.

For example:

```
listaNumeri = [ 1, 2, 5, 7, 8, ]
listaStringhe = [ «Employed", «Unemployed", «Entrepreneur" ]
```

© 2025 – Prof. Emiliano del Gobbo

Object variable

Objects are a relatively complex type of variable, and their nature can vary significantly depending on the programming language.

An object can be considered as a variable that possesses specific properties beyond those of a regular variable.

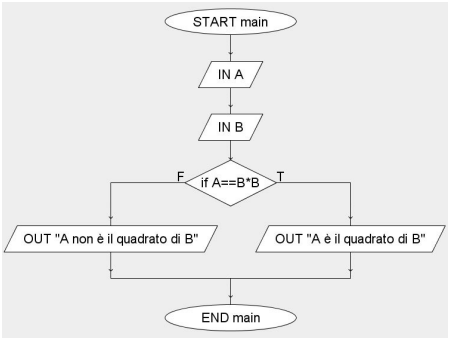
Typically, it is a variable that contains other variables within it.

```
person = {
    name: "Luca"
    surname: "Medici"
    age: 23
    work: "Actor"
    hobby: [ "climbing", "surfing" ]
}
```

© 2025 – Prof. Emiliano del Gobbo

Conditional statement

A **conditional statement** modifies the flow of a program. The outcome of the condition is central: if it evaluates as true, certain operations are executed instead of others.



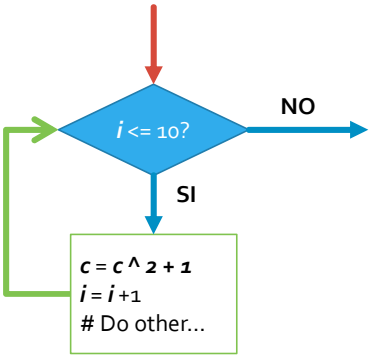

```
if ( a == b ) {
    c = a + 8
} else {
    c = a + ( b / 2 ) +8
}
```


Cycles

A **conditional statement** alters the flow of a program. Central to its operation is the **condition's outcome**, which, if satisfied, leads to the execution of certain operations instead of others.

```
c = 2
i = 1
while ( i <= 10) {
    c = c ^ 2 + 1
    i = i +1
}
```

```
c = 2
for ( i  in 1:10) {
    c = c ^ 2 + 1
}
```



Arithmetic operators

Essential for a **statistical programming language** is the knowledge of **arithmetic operators**, which are largely similar to those used in standard mathematics.

- ***a* + *b*** -> sums ***a*** and ***b***.
- ***a* - *b*** -> subtracts ***b*** to ***a***.
- ***a**b*** -> multiplies ***a*** by ***b***.
- ***a*/ b**-> divides ***a*** by ***b***, returning a decimal number.
- ***a* %/% b**-> divides ***a*** by ***b***, returning only the quotient.
- ***a* %% b** -> divides ***a*** per ***b***, returning only the remain.

© 2025 – Prof. Emiliano del Gobbo

Comparison Operators

Essential for the use of **conditional statements** are **comparison operators**, which are primarily used to determine the flow of a program.

We can perform several types of comparisons:

- ***a* == *b*** -> check if ***a*** and ***b*** are equal.
- ***a* != *b*** -> check if ***a*** e ***b*** are different.
- ***a* > *b*** -> check if ***a*** is greater than ***b***.
- ***a* >= *b*** -> check if ***a*** is greater or equal ***b***.
- ***a* < *b*** -> check if ***a*** is less than ***b***.
- ***a* <= *b*** -> check if ***a*** is less or equal ***b***.

Note: Not all types of variables can use all operators. For example, a string cannot be compared as greater or smaller than another string..

© 2025 – Prof. Emiliano del Gobbo

Functions

A **function** is an algorithm encapsulated in a "box," which, when provided with an input, returns an output.
Essentially, it is a useful way to encapsulate a portion of an algorithm that is frequently used, avoiding the need to rewrite it each time.
In our case, functions are particularly useful because many operations in the course rely on algorithms already implemented by others, which are provided to us as ready-to-use functions within **libraries**.

```
computeCilinderVolume ( radius, height ) {
    baseCilinderArea = 3.14 * raius^2
    volume = baseCilinderArea * height
    return volume
}

smallCilinder = computeCilinderVolume(3, 7)
bigCilinder = computeCilinderVolume(12, 34)
```

© 2025 – Prof. Emiliano del Gobbo

https://bit.ly/424xnaQ

SCAN ME

© 2025 – Prof. Emiliano del Gobbo